What is a Process?

A **process** is a fundamental concept in computing and operating systems (OS). It is the basic unit of execution that allows a computer to perform tasks. To understand a process, let's break it down step-by-step for clarity.

Key Definitions and Concepts

- **Program under execution**: A process is a program that is actively running on a computer. A program is a static set of instructions (like a recipe), while a process is the act of executing those instructions (like cooking the recipe).
- **Program + runtime activity = process**: A process includes not just the program's code but also its runtime environment, such as the data it's working with, the state of the CPU, and the resources it's using (e.g., memory, files).
- **Instruction (code) + operands & other info = process**: A process consists of the program's instructions (the code), the data it operates on (operands), and additional information like the current state of execution.
- **An instance of a program**: If you run the same program multiple times, each running instance is a separate process. For example, opening two web browsers creates two distinct processes, even though they're running the same program.
- **Schedule/dispatchable unit (CPU)**: The operating system uses processes as units that can be scheduled to run on the CPU. The OS decides which process gets CPU time and when.
- **Unit of execution (CPU)**: A process is the entity that the CPU executes. It represents a task that needs processing power.
- **Locus of control (OS)**: A process is the point of control within the operating system, meaning it's where the OS manages and tracks the execution of tasks.

Simple Analogy

Think of a process like a chef in a kitchen. The program is the recipe book (instructions), and the process is the chef actively cooking a dish, using ingredients (data), tools (CPU,

memory), and keeping track of progress (state).

Process as a Data Structure

To manage processes, the operating system treats each process as a **data structure**. A data structure is a way to organize and store data so it can be used efficiently. Let's explore what this means for a process.

Recap: Data Structure

A **data structure** is a format for organizing, storing, and managing data in a computer. Examples include arrays, lists, and tables. In the context of an OS, a process is represented as a data structure to keep track of all the information needed to manage it.

Process as a Data Structure in Depth

The operating system creates a data structure for each process to store all relevant information, such as its current state, resources, and execution details. This data structure is called the **Process Control Block (PCB)** or **Process Descriptor**.

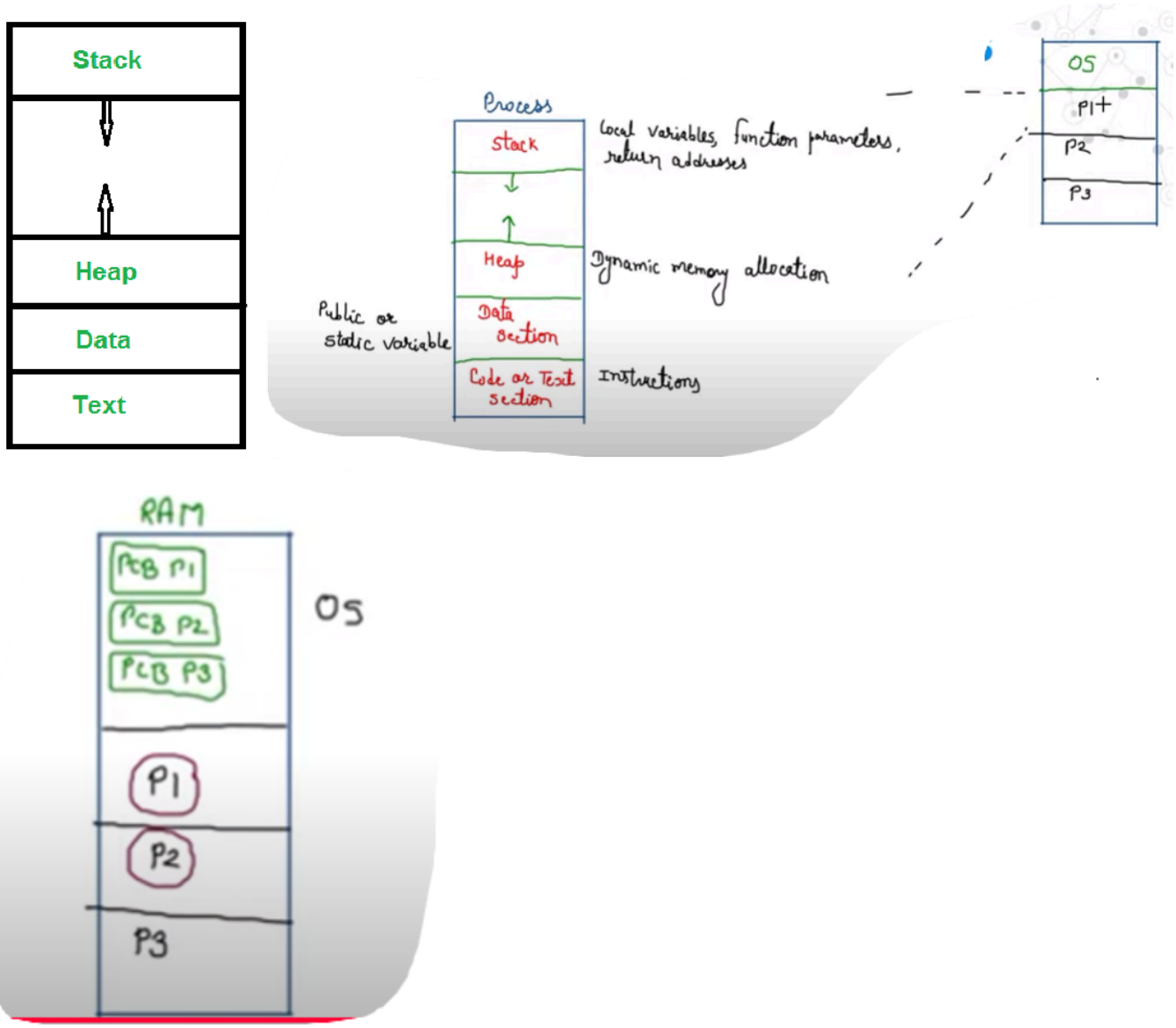
Definition

The PCB is a data structure maintained by the OS for each process. It acts like a "profile" for the process, containing all the information needed to manage and execute it.

Representation/Implementation

A process is represented in memory with several sections, each serving a specific purpose. These sections are part of the process's memory layout and are tracked in the

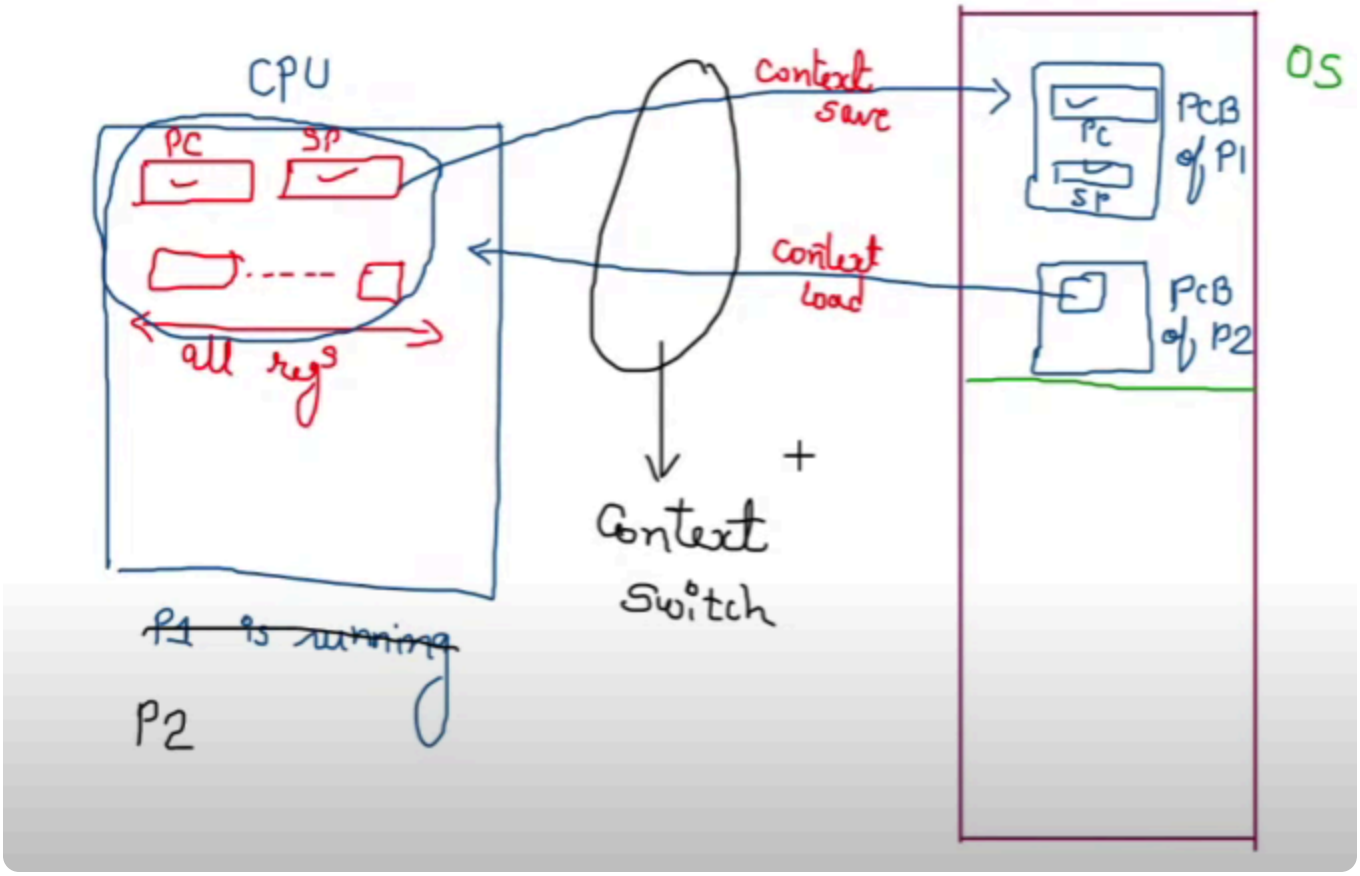
PCB.



PCB is being sorted in the OS part which let it control the resources

- **Stack**:
  - **Purpose**: Used for **static memory allocation**, which means memory is allocated at compile time.
  - **Contents**: Stores local variables, function parameters, and return addresses for function calls.
  - **Example**: When a function is called, its variables and the address to return to after execution are stored in the stack.
  - **Behavior**: The stack grows and shrinks as functions are called and completed. It operates in a Last-In-First-Out (LIFO) manner.
- **Heap**:

    - **Purpose**: Used for **dynamic memory allocation**, where memory is allocated or deallocated during runtime.
    - **Contents**: Stores data like dynamically created objects (e.g., arrays or objects in languages like C++ or Java).
    - **Example**: If a program creates a new list whose size isn't known at compile time, the memory for that list is allocated in the heap.
    - **Behavior**: The heap is more flexible but can lead to fragmentation if memory isn't managed properly.
- **Data Section**:
    - **Purpose**: Stores **global** or **static variables** that are accessible throughout the program's lifetime.
    - **Contents**: Variables declared as static or global (e.g., in C, `static int x = 10;`).
    - **Example**: A counter variable shared across multiple functions would reside here.
- **Code or Text Section**:
    - **Purpose**: Contains the **executable instructions** of the program.
    - **Contents**: The machine code that the CPU executes.
    - **Example**: The compiled code for a function like `print("Hello, World!")` is stored here.
    - **Behavior**: This section is read-only to prevent accidental modification of the program's instructions.



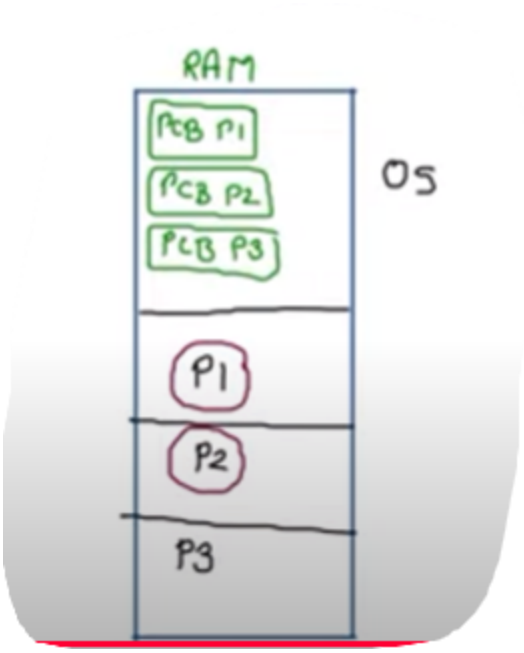
Operations

The OS performs several operations on a process to manage its lifecycle:

- **Create (Resource Allocation)**:
  - The OS creates a new process by allocating resources like memory, CPU time, and I/O devices.
  - A new PCB is initialized to store the process's details.
  - Example: When you open a text editor, the OS creates a process for it.
- **Schedule, Run**:
  - The OS schedules the process to run on the CPU based on its priority and state.
  - The process moves from a ready state to a running state when it gets CPU time.
- **Wait/Block**:
  - A process may need to wait for an event, like user input or I/O completion.
  - The OS moves the process to a blocked state and updates the PCB.
- **Suspend, Resume**:

    - A process can be temporarily suspended (paused) to free up resources for other processes.
    - Resuming a process restores it to a ready or running state.
- **Terminate (Resource Deallocation)**:
    - When a process finishes or is killed, the OS deallocates its resources (e.g., memory, files).
    - The PCB is removed or marked as terminated.

Attributes



The PCB stores various **attributes** of a process to manage its execution:

- **PID (Process ID)**:
    - A unique identifier assigned to each process (e.g., 1234).
    - Used by the OS to distinguish between processes.
- **PC (Program Counter)**:
    - Stores the memory address of the next instruction to be executed.
    - Example: If a process is interrupted, the PC saves where to resume.
- **GPR (General Purpose Registers)**:
    - Temporary storage for data used by the CPU during execution.
    - Example: Registers like AX, BX in x86 architecture hold intermediate values.
- **List of Devices**:

    - Tracks devices (e.g., keyboard, disk) the process is using.
    - Example: A process reading from a file has the file's device listed.
- **Type**:
    - Indicates the type of process (e.g., system process, user process).
    - Example: A background system process vs. a user's web browser.
- **Size**:
    - The amount of memory allocated to the process.
    - Example: A large program like a video editor may require more memory than a calculator app.
- **Memory Limit**:
    - The maximum memory the process is allowed to use.
    - Prevents a process from consuming too many resources.
- **Priority**:
    - Determines the process's scheduling priority.
    - Example: A critical system process may have higher priority than a user app.
- **State**:
    - The current state of the process (e.g., New, Ready, Running, Waiting, Terminated).
    - Tracks where the process is in its lifecycle.
- **List of Files**:
    - Tracks open files associated with the process.
    - Example: A text editor process keeps track of the file being edited.

Context

The **context** of a process refers to the complete set of information stored in its PCB at any given time. It includes all attributes (like PID, PC, registers) and represents the process's current state. The context allows the OS to pause a process, save its state, and resume it later without losing progress.

Context Switch, Context Save, and Context Load

Context Switch

A **context switch** is the process of switching the CPU from executing one process (e.g., P1) to another (e.g., P2). This happens frequently in a multitasking OS to share CPU time among multiple processes.

- **Why it happens**:
  - The OS uses a scheduler to decide which process runs next based on priorities, time slices, or events (e.g., a process waiting for I/O).
  - Example: If P1 is waiting for user input, the OS switches to P2 to keep the CPU busy.
- **How it works**:
  1. The OS interrupts the currently running process (P1).
  2. The **context save** happens: The OS saves P1's current state (all PCB attributes, including registers, PC, etc.) to its PCB in RAM.
  3. The **context load** happens: The OS loads P2's context (from its PCB) into the CPU, including its registers, PC, and other attributes.
  4. The CPU resumes execution of P2 from where it left off.
- **Role of the Dispatcher**:
  - The **dispatcher** is a special OS program responsible for performing context switches.
  - It saves the context of the current process, updates the PCB, and loads the context of the next process.
  - The dispatcher ensures smooth transitions between processes without data loss.

Context Save

- **Definition**: The process of saving a process's current state to its PCB in RAM when it is interrupted or preempted.
- **What is saved**:
  - **Program Counter (PC)**: The address of the next instruction.

    - **Registers**: All CPU registers (e.g., general-purpose registers like AX, BX, and special registers like stack pointer).
    - **Process State**: The current state (e.g., Running to Ready or Waiting).
    - **Other Attributes**: Memory pointers, open files, and device statuses.
- **Example**:
    - P1 is running a loop and is interrupted. The OS saves P1's PC (pointing to the next loop instruction), register values (e.g., loop counter), and other details to P1's PCB in RAM.

Context Load

- **Definition**: The process of loading a process's saved state from its PCB into the CPU to resume execution.
- **What is loaded**:
    - The saved PC, registers, and other attributes from the PCB are copied into the CPU.
    - The OS updates the process state (e.g., Ready to Running).
- **Example**:
    - When P2 is scheduled, the OS loads P2's saved PC, registers, and other details from its PCB into the CPU, allowing P2 to pick up where it left off.

Deep Dive: How CPU Saves Information for P1 to PCB

When the OS decides to switch from P1 to P2, the following steps occur:

1. **Interrupt**: The CPU receives an interrupt (e.g., timer interrupt for time-sharing or I/O completion).
2. **Save Registers**:
    - The CPU's registers (e.g., general-purpose registers, stack pointer, flags) are saved to P1's PCB in RAM.
    - Example: If P1 was performing a calculation, the intermediate result in a register (e.g., AX = 42) is saved.
3. **Save Program Counter**:
    - The PC, which points to the next instruction, is saved to P1's PCB.

- Example: If P1 was at instruction 1005, the PC value 1005 is stored.

4. **Update Process State**:
   - P1's state is updated in the PCB (e.g., from Running to Ready or Waiting).
5. **Store Other Attributes**:
   - Memory pointers, open file descriptors, and device statuses are updated in P1's PCB.
6. **Load P2's Context**:
   - The dispatcher loads P2's PCB data (PC, registers, etc.) into the CPU.
7. **Resume Execution**:
   - The CPU starts executing P2's instructions from the saved PC value.

This process ensures that P1's execution state is preserved and can be restored later, allowing seamless multitasking.

Questions and Answers with Elaboration

Q1: While Running, a process can access its PCB from main memory?

Answer: **False**. The PCB of all processes is stored in a **protected area of memory** managed by the operating system. A process cannot directly access its own PCB or any other process's PCB.

- **Elaboration**:
  - The PCB contains critical information about a process, such as its state, registers, and resource allocations. Allowing a process to access its PCB could lead to security risks or unintended modifications.
  - The OS maintains strict control over PCBs, which reside in a kernel-protected memory region. Only the OS (via kernel-mode operations) can read or modify PCBs.

- Example: If a process could modify its own priority in the PCB, it might unfairly monopolize CPU time, disrupting the system.

Q2: A Process in the context of computing is:

a. A set of instructions to be executed on a computer
b. A program in execution
c. A piece of hardware that executes a set of instructions
d. The main procedure of a program

Answer: **b. A program in execution**

- **Elaboration**:
  - **Option a**: A set of instructions is a **program**, not a process. A process is the program plus its runtime state.
  - **Option b (Correct)**: A process is a program in execution, including its code, data, and state (e.g., registers, PC).
  - **Option c**: Hardware (like the CPU) executes processes, but a process itself is not hardware.
  - **Option d**: The main procedure is part of a program's code, not the entire process.
  - Example: When you run a web browser, the browser's code (program) becomes a process as it executes, using memory, CPU, and other resources.

Q3: Which technique was introduced because a single job could not keep both CPU and I/O devices busy?

a. Real Time
b. Spooling
c. Preemptive Scheduling
d. Multiprogramming

Answer: **d. Multiprogramming**

- **Elaboration**:
  - **Why Multiprogramming?**:

        - In early computers, a single job (process) would often leave the CPU or I/O devices idle. For example, while a process waits for I/O (like reading a file), the CPU is idle.
        - **Multiprogramming** allows multiple processes to reside in memory simultaneously. When one process waits for I/O, the OS switches to another process, keeping the CPU busy.
    - **Other Options**:
        - **Real Time**: Focuses on executing tasks with strict timing constraints, not on keeping CPU and I/O busy.
        - **Spooling**: Buffers I/O operations (e.g., for printers) to reduce wait times but doesn't address CPU utilization directly.
        - **Preemptive Scheduling**: Allows the OS to interrupt a running process to run another, but it's a scheduling technique, not the concept introduced to solve CPU/I/O idleness.
    - **Example**: In a multiprogramming system, while one process waits for a file to load, another process (e.g., a text editor) can use the CPU, improving efficiency.

Preemptive Scheduling

What is Preemptive Scheduling?

Preemptive scheduling is a technique used by the operating system to manage CPU allocation among processes. In this approach, the OS can interrupt (preempt) a running process to allocate CPU time to another process, even if the current process hasn't finished.

- **How it works**:
    - The OS uses a scheduler to decide which process runs next based on factors like priority, time slices, or events.
    - A timer interrupt periodically triggers the scheduler to check if a higher-priority process is ready to run.

    - If a higher-priority process is ready, the OS performs a context switch, saving the current process's state and loading the new process's state.
- **Key Features**:
    - **Time Slicing**: Each process gets a fixed time slot (quantum) to run before being preempted.
    - **Priority-Based**: Higher-priority processes can preempt lower-priority ones.
    - **Responsiveness**: Ensures quick response to critical tasks or user interactions.
    - **Fairness**: Prevents any single process from monopolizing the CPU.
- **Example**:
    - Suppose P1 is running a long computation, and a user clicks the mouse, triggering P2 (a user interface process). The OS preempts P1, saves its context, and runs P2 to handle the mouse click, ensuring the system remains responsive.
- **Advantages**:
    - Improves system responsiveness, especially in interactive systems (e.g., GUI-based OS).
    - Ensures fair CPU sharing among processes.
    - Supports real-time systems where urgent tasks need immediate attention.
- **Disadvantages**:
    - Overhead from frequent context switches (saving and loading PCB data).
    - Complex to implement compared to non-preemptive scheduling.
- **Comparison with Non-Preemptive Scheduling**:
    - In **non-preemptive scheduling**, a process runs until it completes or voluntarily yields the CPU (e.g., by waiting for I/O).
    - Preemptive scheduling is more dynamic and suited for modern multitasking OS like Windows, Linux, or macOS.

Summary

- A **process** is a program in execution, managed by the OS as a data structure called the **PCB**.

- The PCB stores attributes like PID, PC, registers, and state, collectively called the **context**.
- **Context switching** allows the OS to switch between processes (e.g., P1 to P2) by saving and loading their contexts using the **dispatcher**.
- Operations like create, schedule, wait, suspend, and terminate manage a process's lifecycle.
- **Preemptive scheduling** enhances multitasking by allowing the OS to interrupt processes, ensuring efficient CPU and I/O utilization.
- **Multiprogramming** was introduced to keep CPU and I/O devices busy by running multiple processes concurrently.